

ROADMAP MASTER

MODULO 2 — dbt: Tu ETL con control de versiones

Version 1 DEFINITIVA · Semanas 6 y 7 · Nivel: Junior 0 absoluto

Tablas visuales antes/despues + Explicaciones linea por linea + Proyectos 3 y 5

Duracion	Semanas	Proyectos	Prerequisito
2 semanas	6 y 7	P3: Data Mart + P5: DWH Cloud	Modulos 0, 1A y 1B completados

Lo que aprenderas en este modulo
★ Por que el ETL manual de PL/pgSQL que hiciste en 1B no escala — y como dbt lo resuelve
★ Como instalar y configurar dbt Core con tu PostgreSQL local en Docker
★ Que son los modelos dbt y como reemplazar tus queries SQL con archivos .sql versionados
★ Las 4 capas de un proyecto dbt profesional: sources, staging, intermediate, marts
★ Como agregar tests automaticos que validan tus datos en cada ejecucion
★ Como generar documentacion visual con linaje automatico con dbt docs
★ Proyecto 3: Data Mart completo sobre los datos de tienda con dbt
★ Proyecto 5: El mismo proyecto corriendo en BigQuery (free tier)

1. El problema que dbt resuelve — por que existe

1.1 El ETL manual: lo que construiste en el Modulo 1B

En el Modulo 1B construiste un stored procedure ETL manual en PL/pgSQL. Funciona para aprender. Pero en produccion con un equipo real tiene problemas serios.

LO QUE CONSTRUISTE EN 1B — el ETL manual

Componente	Como lo hiciste	El problema en produccion
La transformacion	SQL dentro de un PROCEDURE en PostgreSQL	No esta en Git. No se puede revisar en un PR. No se puede testear en CI/CD.
La documentacion	Comentarios en el codigo SQL	Si cambias la logica, los comentarios quedan desactualizados. Nadie sabe que hace realmente.
Los tests	Ninguno — confias en que el SQL es correcto	Un cambio rompe algo y no te enteras hasta que el CEO llama.
El linaje	Tu sabes de donde viene cada campo — pero nadie mas	Cuando alguien nuevo entra al equipo, no sabe que tablas dependen de que.
La ejecucion	CALL tienda.etl_cargar_ventas() manualmente	Hay que recordar cuando y como ejecutarlo. Propenso a errores humanos.

1.2 La solucion: dbt transforma tu SQL en un pipeline profesional

CON DBT — los mismos problemas resueltos

Componente	Como lo hace dbt	Beneficio real
La transformacion	Archivos .sql en una carpeta — uno por modelo	Cada modelo es un archivo en Git. Se revisa en PR. Se ve el historial de cambios.
La documentacion	Archivo .yaml con descripcion de cada modelo y cada columna	dbt genera un sitio web automatico con toda la documentacion siempre actualizada.
Los tests	Tests declarativos en YAML: not_null, unique, relationships, accepted_values	dbt test falla el build si un dato no cumple las reglas. CI/CD lo detecta automatico.
El linaje	dbt genera un grafo visual de dependencias automaticamente	Cualquier persona del equipo puede ver de donde viene cada campo en 2 clicks.
La ejecucion	dbt run — un solo comando ejecuta todo en el orden correcto	Siempre el mismo comando. Se puede agregar a Airflow o GitHub Actions.

DBT EN UNA FRASE

Sin dbt	Con dbt
---------	---------

SELECT ... INTO tabla_resultado; (SQL ejecutado manualmente o en un PROCEDURE sin control)	Un archivo modelos/marts/ventas_por_mes.sql con el mismo SELECT. dbt run lo ejecuta, lo testea y lo documenta automaticamente.
--	---

dbt NO reemplaza SQL. dbt ORGANIZA y AUTOMATIZA tu SQL existente. El SQL que escribiste en 1A y 1B funciona exactamente igual dentro de dbt.

2. Instalacion y configuracion — paso a paso

2.1 Que necesitas antes de empezar

PREREQUISITOS — verificar antes de instalar dbt

Prerequisito	Como verificar	Version minima
Docker corriendo con curso_postgres	<code>docker ps -> ver curso_postgres (healthy)</code>	Docker 24+
Python 3.12 instalado	<code>python3 --version -> Python 3.12.x</code>	Python 3.10+
pip actualizado	<code>pip3 install --upgrade pip</code>	pip 23+
Git configurado	<code>git --version -> git version 2.x</code>	Git 2.30+
El repositorio del curso	<code>ls ~/proyectos/data-engineering-master/</code>	Creado en Fase 0

2.2 Instalar dbt-core y dbt-postgres

```
# Paso 1: crear un entorno virtual para dbt (buena practica)
cd ~/proyectos/data-engineering-master
python3 -m venv venv-dbt
source venv-dbt/bin/activate
# (en Windows: venv-dbt\Scripts\activate)

# Paso 2: instalar dbt-core y el adaptador de PostgreSQL
pip install dbt-core dbt-postgres

# Verificar la instalacion:
dbt --version
# Debe mostrar algo como:
# Core:
#   - installed: 1.7.x
#   - latest:    1.7.x
# Plugins:
#   - postgres: 1.7.x

# Si el comando dbt no se encuentra, agregar al PATH:
export PATH="$HOME/.local/bin:$PATH"
# Agregar esta linea a ~/.bashrc o ~/.zshrc para que persista
```

! Siempre activa el entorno virtual antes de usar dbt

Cada vez que abras una terminal nueva, ejecuta: `cd ~/proyectos/data-engineering-master && source venv-dbt/bin/activate`. Si el entorno no esta activo, dbt no funcionara.

2.3 Crear el proyecto dbt

```
# Paso 3: crear el proyecto dbt
cd ~/proyectos/data-engineering-master
dbt init tienda_dbt
```

```
# dbt preguntara:
# Which database adapter would you like to use?
# [1] postgres
# Escribe: 1 y presiona Enter

# Resultado: se crea la carpeta tienda_dbt/ con esta estructura:
tienda_dbt/
├── dbt_project.yml    <- configuracion del proyecto
├── models/           <- aqui van tus archivos .sql
│   └── example/      <- modelos de ejemplo (los borraremos)
├── tests/            <- tests personalizados
├── macros/           <- funciones reutilizables
├── seeds/            <- datos estaticos en CSV
└── snapshots/        <- historial de cambios (SCD)
```

2.4 Configurar la conexion a PostgreSQL

dbt necesita saber a que base de datos conectarse. Esta configuracion va en un archivo llamado profiles.yml que vive fuera del proyecto (para no subir credenciales a Git).

```
# El archivo profiles.yml se crea en:
# Linux/Mac: ~/.dbt/profiles.yml
# Windows:   C:\Users\TuUsuario\.dbt\profiles.yml

# Crearlo:
mkdir -p ~/.dbt
nano ~/.dbt/profiles.yml

# Contenido del archivo profiles.yml:
tienda_dbt:
  target: dev
  outputs:
    dev:
      type: postgres
      host: localhost
      port: 5432
      user: admin
      password: masterpass2025
      dbname: cursodb
      schema: dbt_dev
      # dbt creara automaticamente el schema dbt_dev en cursodb
      # Todas las tablas y vistas que genere dbt iran aqui
      threads: 4
      # Cuantos modelos puede ejecutar dbt en paralelo

# Verificar la conexion:
cd ~/proyectos/data-engineering-master/tienda_dbt
dbt debug
# Debe mostrar: All checks passed!
```

RESULTADO de dbt debug cuando esta bien configurado

Verificacion	Resultado esperado
dbt version	OK found dbt-core==1.7.x
profiles.yml file	OK found it at /home/usuario/.dbt/profiles.yml

profiles.yml valid	OK
credentials	OK
Connection test	OK connection ok

Si alguna verificacion falla, el mensaje de error te dice exactamente que corregir. El mas comun es "Connection refused" — verifica que Docker este corriendo con docker ps.

! profiles.yml NUNCA va en Git

El archivo profiles.yml contiene tu password de la base de datos. Nunca lo subas a GitHub. Agrega .dbt/ a tu .gitignore global. En equipos reales se usan variables de entorno o un vault de secretos.

2.5 Usar variables de entorno — la forma correcta en produccion

El profiles.yml del ejemplo anterior tiene la password en texto plano. En produccion NUNCA se hace eso. Se usan variables de entorno con env_var().

TEXTO PLANO vs VARIABLES DE ENTORNO

Enfoque	Como se ve en profiles.yml	Problema
Texto plano (SOLO para aprender localmente)	password: masterpass2025	Si alguien ve el archivo ve tu password. Si lo subes a Git por accidente, queda expuesto en el historial.
Variable de entorno (produccion)	password: {{ env_var('DBT_DB_PASSWORD') }}	La password no esta en el archivo. Esta en el sistema operativo o en un vault de secretos.

```
# Archivo: ~/.dbt/profiles.yml con variables de entorno

tienda_dbt:
  target: dev
  outputs:
    dev:
      type: postgres
      host: "{{ env_var('DBT_DB_HOST', 'localhost') }}"
      # env_var('VAR', 'default'): usa la variable de entorno DBT_DB_HOST
      # Si no existe, usa el valor por defecto: localhost
      port: "{{ env_var('DBT_DB_PORT', '5432') | int }}"
      # | int: convierte el string a entero (el puerto debe ser numero)
      user: "{{ env_var('DBT_DB_USER') }}"
      # Sin default: si la variable no existe, dbt da error al arrancar
      password: "{{ env_var('DBT_DB_PASSWORD') }}"
      dbname: "{{ env_var('DBT_DB_NAME', 'cursodb') }}"
      schema: dbt_dev
      threads: 4

# Como configurar las variables en tu terminal:

# Linux / Mac (en ~/.bashrc o ~/.zshrc para que persistan):
export DBT_DB_HOST=localhost
export DBT_DB_PORT=5432
```

```
export DBT_DB_USER=admin
export DBT_DB_PASSWORD=masterpass2025
export DBT_DB_NAME=cursodb

# Aplicar los cambios sin reiniciar la terminal:
source ~/.bashrc

# Verificar que la variable existe:
echo $DBT_DB_PASSWORD
# Debe mostrar: masterpass2025

# Windows PowerShell:
$env:DBT_DB_PASSWORD = "masterpass2025"

# En CI/CD (GitHub Actions):
# Se configuran como Secrets en el repositorio
# Settings -> Secrets -> New secret -> DBT_DB_PASSWORD
# GitHub las inyecta automáticamente como variables de entorno
```

Para el curso local puedes usar texto plano

- ! El profiles.yml con password en texto plano esta bien para aprender en local
- ! Cuando el proyecto sea real o lo subas a un servidor: cambia a env_var()
- ! En empresas se usan vaults (HashiCorp Vault, AWS Secrets Manager) con env_var()
- ! La regla de oro: si el archivo puede llegar a Git, la password no debe estar ahí

3. Conceptos core de dbt — explicados desde cero

3.1 Un modelo dbt = un archivo .sql = una tabla o vista

Esta es la idea central de dbt. Cada modelo es un archivo .sql que contiene un SELECT. dbt ejecuta ese SELECT y crea una tabla o vista con el resultado.

LA TRANSFORMACION MENTAL — de PL/pgSQL a dbt

En PL/pgSQL (Modulo 1B)	En dbt (Modulo 2)
Un PROCEDURE con INSERT ... SELECT dentro CALL tienda.etl_cargar_ventas() para ejecutar Existe en la BD, no en archivos	Un archivo models/marts/hechos_ventas.sql con SELECT dbt run para ejecutar Existe en Git, versionado, revisable

```
-- El modelo mas simple posible
-- Archivo: models/marts/resumen_clientes.sql

SELECT
  c.cliente_id,
  c.nombre,
  c.ciudad,
  COUNT(p.pedido_id) AS total_pedidos,
  COALESCE(SUM(p.total), 0) AS total_gastado
FROM tienda.clientes c
LEFT JOIN tienda.pedidos p
  ON c.cliente_id = p.cliente_id
  AND p.estado = 'completado'
WHERE c.activo = TRUE
GROUP BY c.cliente_id, c.nombre, c.ciudad

-- Eso es todo. Solo un SELECT.
-- dbt run crea automaticamente la tabla dbt_dev.resumen_clientes
-- con el resultado de este SELECT.
```

QUE HACE dbt run con ese archivo

Pa so	Que hace dbt	Resultado
1	Lee el archivo resumen_clientes.sql	Sabe que hay un modelo llamado resumen_clientes
2	Detecta que tablas necesita (tienda.clientes, tienda.pedidos)	Construye el orden de ejecucion
3	Ejecuta: CREATE TABLE dbt_dev.resumen_clientes AS (tu SELECT)	Crea la tabla con los datos
4	Registra en logs: OK resumen_clientes (X rows)	Confirma que funciono y cuantas filas tiene

Por defecto dbt crea VISTAS (CREATE VIEW). Para crear tablas fisicas (mas rapidas de consultar) se configura la materializacion.

3.2 ref() — como los modelos se referencian entre si

En dbt, cuando un modelo necesita usar otro modelo, no escribes el nombre de la tabla directamente. Usas la funcion `ref()`. Esto le dice a dbt "este modelo depende del otro" y construye el grafo de dependencias automaticamente.

SIN ref() — el problema

Modelo A	Modelo B (usa A)	El problema
models/staging/ stg_pedidos.sql crea: dbt_dev.stg_pedidos	SELECT * FROM dbt_dev.stg_pedidos (nombre hardcodedo)	Si cambias el schema de dev a prod, el modelo B se rompe. dbt no sabe que B depende de A.

CON ref() — la solucion

Modelo A	Modelo B con ref()	El beneficio
models/staging/ stg_pedidos.sql crea: dbt_dev.stg_pedidos	SELECT * FROM {{ ref('stg_pedidos') }} (ref dinamico)	dbt resuelve el nombre correcto automaticamente. Construye el linaje. Ejecuta A antes que B siempre.

```
-- Ejemplo: modelo que usa ref()
-- Archivo: models/marts/ventas_por_mes.sql

SELECT DATE_TRUNC('month', fecha)::DATE AS mes,
       COUNT(pedido_id)                  AS num_pedidos,
       SUM(total)                        AS ventas_totales
FROM {{ ref('stg_pedidos') }}
-- {{ ref("stg_pedidos") }}: dbt reemplaza esto por el nombre real de la tabla
-- Cuando corres en dev: dbt_dev.stg_pedidos
-- Cuando corres en prod: dbt_prod.stg_pedidos
-- dbt sabe que debe ejecutar stg_pedidos ANTES que ventas_por_mes
WHERE estado = 'completado'
GROUP BY DATE_TRUNC('month', fecha)
ORDER BY mes;
```

3.3 source() — referenciar tablas que NO son modelos dbt

Las tablas de tienda (clientes, pedidos, etc.) no son modelos dbt — son datos que ya existen en la base de datos. Para referenciarlas de forma segura usas `source()`.

```
-- Primero: declarar las fuentes en un archivo .yaml
-- Archivo: models/staging/sources.yaml

version: 2

sources:
  - name: tienda
    schema: tienda
    description: "Tablas de la base de datos de la tienda"
    tables:
      - name: clientes
```

```

    description: "Clientes activos e inactivos de la tienda"
    columns:
      - name: cliente_id
        description: "Identificador unico del cliente"
      - name: pedidos
        description: "Todos los pedidos realizados"
      - name: productos
        description: "Catalogo de productos"
      - name: detalle_pedidos
        description: "Lineas de detalle de cada pedido"

-- Luego: usar source() en los modelos de staging
-- Archivo: models/staging/stg_clientes.sql

SELECT
  cliente_id,
  nombre,
  email,
  ciudad,
  pais,
  fecha_reg,
  activo
FROM {{ source('tienda', 'clientes') }}
-- {{ source("tienda", "clientes") }}: dbt lo reemplaza por tienda.clientes
-- Beneficio: dbt puede hacer freshness checks sobre los datos de origen

```

3.4 Las 4 capas de un proyecto dbt profesional

En un proyecto dbt bien organizado, los modelos se agrupan en capas. Cada capa tiene una responsabilidad específica.

LAS 4 CAPAS — de los datos crudos hasta el reporte final

Capa	Carpeta	Que hace	Ejemplo de modelo	Materializaci3n
1. Sources	sources.yml	Declara las tablas de origen que ya existen. No crea nada.	tienda.clientes, tienda.pedidos	(ninguna — solo declaraci3n)
2. Staging	models/staging/	Limpia y estandariza los datos crudos. Un modelo por tabla de origen. Renombra columnas, castea tipos, filtra filas invalidas.	stg_clientes.sql stg_pedidos.sql	view (liviano)
3. Intermediate	models/intermediate/	Combina modelos de staging. Aplica logica de negocio. Puede ser complejo pero reutilizable.	int_pedidos_con_cliente.sql	view o ephemeral
4. Marts	models/marts/	Los modelos finales que los analistas y dashboards consumen. Deben ser rapidos. Logica de negocio ya aplicada.	ventas_por_mes.sql resumen_clientes.sql	table (rapido)

Esta estructura es la convencion del equipo de dbt. No es obligatoria, pero la adoptan el 90% de los equipos profesionales.

FLUJO VISUAL DE LAS CAPAS

Origen		Staging		Intermediate		Marts		Consumo
tienda.clientes	—	stg_clientes	—	int_pedidos_compl	—	ventas_por_mes	—	Dashboard
tienda.pedidos	—	stg_pedidos	—	etos	—	resumen_clientes	—	Reportes
tienda.productos	►	stg_productos	►	int_clientes_con_ga	►	rfm_segmentacion	►	API de analytics
tienda.detalle		stg_detalle		sto				

Los datos fluyen de izquierda a derecha. Cada capa agrega valor. dbt construye el orden de ejecucion automaticamente usando ref() y source().

4. Materializaciones — tabla, vista, incremental, ephemeral

La materializacion define como dbt persiste el resultado de un modelo. Elegir la materializacion correcta es clave para el rendimiento.

LAS 4 MATERIALIZACIONES — cuando usar cada una

Tipo	Que crea en la BD	Velocidad de lectura	Costo de dbt run	Usa cuando...
view	CREATE VIEW — una query guardada, no datos fisicos	Lenta (re-ejecuta la query cada vez)	Muy bajo (solo guarda la definicion)	Staging. Los datos cambian frecuentemente. No se consulta muchas veces.
table	CREATE TABLE — datos fisicos en disco	Rapida (los datos ya estan calculados)	Medio (recrea la tabla entera cada vez)	Marts. Modelos finales que los dashboards consultan miles de veces.
incremental	INSERT solo las filas NUEVAS desde la ultima ejecucion	Rapida	Bajo (solo procesa datos nuevos)	Tablas grandes donde solo llegan datos nuevos. Como hechos_ventas.
ephemeral	No crea nada — es como un CTE dentro de otro modelo	N/A (no existe como objeto)	Cero (se incrusta en el modelo que lo usa)	Logica intermedia que solo necesita un modelo. No quieres llenar la BD de objetos.

```
-- Opcion 1: configurar en el archivo dbt_project.yml (para toda una carpeta)
models:
  tienda_dbt:
    staging:
      +materialized: view # todos los modelos de staging son vistas
    intermediate:
      +materialized: view
    marts:
      +materialized: table # todos los marts son tablas fisicas

-- Opcion 2: configurar en el propio archivo .sql del modelo
-- Archivo: models/marts/ventas_por_mes.sql

{{
  config(
    materialized='table',
    description='Ventas agrupadas por mes, solo pedidos completados'
  )
}}

SELECT ...
-- El bloque config() va al inicio del archivo, antes del SELECT
```

4.1 Modelo incremental — para tablas grandes

El modelo incremental es el mas importante para pipelines de datos reales. En lugar de recrear toda la tabla en cada ejecucion, solo inserta las filas nuevas.

COMPARACION: table vs incremental con 1 millon de pedidos

Ejecucion	Materializacion table	Materializacion incremental
Primera ejecucion	Procesa 1M pedidos -> crea tabla (3 minutos)	Procesa 1M pedidos -> crea tabla (3 minutos)
Segunda ejecucion (100 pedidos nuevos)	Procesa TODOS los 1M pedidos de nuevo (3 minutos)	Procesa solo los 100 pedidos nuevos (1 segundo)
Decimosegunda ejecucion	Siempre 3 minutos aunque no haya nada nuevo	Siempre proporcional a los datos nuevos

Para la base de datos del curso con 10 pedidos la diferencia es insignificante. Con datos reales de produccion, incremental puede ser 100x mas rapido.

```
-- Modelo incremental para hechos_ventas
-- Archivo: models/marts/hechos_ventas.sql

{{
    config(
        materialized='incremental',
        unique_key='hecho_id'
        -- unique_key: si ya existe una fila con ese ID, la actualiza en lugar de duplicar
    )
}}

SELECT
    p.pedido_id || '-' || dp.detalle_id AS hecho_id,
    -- Clave unica: combinacion de pedido + detalle
    p.fecha::DATE AS fecha,
    p.cliente_id,
    dp.producto_id,
    p.pedido_id,
    dp.cantidad,
    dp.cantidad * dp.precio_unit AS subtotal
FROM {{ source('tienda', 'pedidos') }} p
JOIN {{ source('tienda', 'detalle_pedidos') }} dp
    ON p.pedido_id = dp.pedido_id
WHERE p.estado = 'completado'

{% if is_incremental() %}
    -- Este bloque SOLO se ejecuta en ejecuciones incrementales (no en la primera)
    AND p.fecha > (SELECT MAX(fecha) FROM {{ this }})
    -- {{ this }}: referencia a la tabla que ya existe (hechos_ventas)
    -- Solo trae pedidos mas recientes que el ultimo que ya tenemos
{% endif %}
```

! unique_key con concatenacion — advertencia para tablas grandes

El ejemplo usa `pedido_id || "-" || detalle_id` como `unique_key`. Funciona correctamente pero tiene un costo: dbt hace un MERGE/UPDATE comparando strings en lugar de enteros, lo que es mas lento en tablas grandes. En produccion con millones de filas, usa siempre una columna numerica como `unique_key`. Alternativa: agregar una columna `hecho_id BIGSERIAL` a la tabla de destino y usar esa como `unique_key`.

4.2 on_schema_change — que pasa cuando cambia la estructura

Cuando modificas las columnas de un modelo incremental (agregas o eliminas columnas), dbt necesita saber que hacer con la tabla existente. Eso lo controla on_schema_change.

OPCIONES DE on_schema_change

Valor	Que hace cuando cambia el schema	Cuando usarlo
ignore (default)	Ignora los cambios. Las columnas nuevas no se agregan. Las eliminadas quedan.	Cuando los cambios de schema son raros y prefieres control manual
fail	Da error si detecta un cambio de schema. Fuerza al developer a tomar una decision.	En produccion donde cualquier cambio de schema debe ser intencional y revisado
append_new_columns	Agrega las columnas nuevas a la tabla existente. No borra las eliminadas.	Cuando agregas columnas frecuentemente pero no quieres perder datos
sync_all_columns	Sincroniza todas las columnas: agrega las nuevas, elimina las que ya no estan.	Con cuidado — puede borrar columnas con datos

```
-- Configurar on_schema_change en el modelo incremental:
{{
  config(
    materialized='incremental',
    unique_key='pedido_id',
    on_schema_change='append_new_columns'
    -- Si agregas una columna al SELECT, la agrega a la tabla existente
    -- Sin borrar los datos que ya habia
  )
}}

SELECT ...
```

4.3 dbt run --full-refresh — cuando necesitas reconstruir todo

Cuando cambias la logica de un modelo incremental (no solo el schema), la tabla existente tiene datos calculados con la logica ANTIGUA. Necesitas reconstruir toda la tabla desde cero.

CUANDO NECESITAS --full-refresh

Situacion	Sin --full-refresh	Con --full-refresh
Cambias el WHERE de un modelo incremental	Solo aplica el nuevo WHERE a los datos nuevos. Los datos viejos mantienen el WHERE antiguo. Inconsistente.	Borra la tabla y la recrea entera con el nuevo WHERE. Consistente.
Agregas una columna calculada	La columna nueva existe pero los registros viejos tienen NULL en ella.	Reconstruye todo. Todos los registros tienen la columna calculada.

Corriges un bug en la logica	El bug queda en los datos historicos. Solo los nuevos son correctos.	Reconstruye todo. Todos los registros son correctos.
------------------------------	--	--

```
# Reconstruir un modelo incremental desde cero:
dbt run --select hechos_ventas --full-refresh
# Equivale a DROP TABLE + CREATE TABLE (no INSERT)
# Todos los datos se recalculan desde el principio

# Reconstruir TODOS los modelos desde cero:
dbt run --full-refresh
# Util cuando cambias la logica de varios modelos a la vez

# CUIDADO en produccion:
# --full-refresh en una tabla con millones de filas puede tardar horas
# Planificalo en una ventana de mantenimiento (fuera de horario de negocio)

# Ver si un modelo necesita full-refresh:
# Si dbt run da error: "Cannot use is_incremental() with --full-refresh"
# o los datos no tienen sentido, es senal de que necesitas --full-refresh
```

5. Construir el proyecto de la tienda — paso a paso

5.1 Estructura del proyecto que vamos a construir

```
tienda_dbt/
├── dbt_project.yml
├── models/
│   ├── staging/
│   │   ├── sources.yml          <- declara tienda.clientes, pedidos, etc.
│   │   ├── stg_clientes.sql     <- limpia y estandariza clientes
│   │   ├── stg_pedidos.sql      <- limpia y estandariza pedidos
│   │   ├── stg_productos.sql    <- limpia y estandariza productos
│   │   ├── stg_detalle.sql      <- limpia y estandariza detalle_pedidos
│   │   └── schema.yml           <- tests y documentacion de staging
│   ├── intermediate/
│   │   ├── int_pedidos_completos.sql <- pedidos con info del cliente
│   │   └── schema.yml
│   └── marts/
│       ├── ventas_por_mes.sql
│       ├── resumen_clientes.sql
│       ├── top_productos.sql
│       ├── rfm_segmentacion.sql
│       ├── hechos_ventas.sql     <- modelo incremental (Proyecto 3)
│       └── schema.yml
└── tests/
    └── assert_ventas_positivas.sql <- test personalizado
```

5.2 dbt_project.yml — la configuracion del proyecto

```
# Archivo: tienda_dbt/dbt_project.yml
# (reemplaza el contenido por defecto)

name: tienda_dbt
version: "1.0.0"
config-version: 2

profile: tienda_dbt
# Debe coincidir con el nombre en ~/.dbt/profiles.yml

model-paths: ["models"]
test-paths: ["tests"]
seed-paths: ["seeds"]
macro-paths: ["macros"]

models:
  tienda_dbt:
    staging:
      +materialized: view
      # Los modelos de staging son vistas (livianos)
    intermediate:
      +materialized: view
    marts:
      +materialized: table
      # Los modelos de marts son tablas fisicas (rapidas)
```

5.3 Capa Staging — limpiar y estandarizar

La capa staging toma los datos de origen tal como estan y los limpia. Un modelo de staging por tabla de origen. Solo limpieza, sin logica de negocio.

DATOS DE ORIGEN: tienda.clientes (tal como estan)

cliente_id	nombre	email	ciudad	pais	fecha_reg	act
1	Ana Torres	ana@email.com	Lima	Peru	2024-01-01	TRUE
2	Luis Garcia	luis@email.com	Bogota	Colombia	2024-01-05	TRUE
8	Juan Perez	juan@email.com	Lima	Peru	2024-02-20	TRUE

El staging no cambia los datos. Solo los renombra, castea tipos y filtra invalidos.

```
-- Archivo: models/staging/stg_clientes.sql

SELECT
  cliente_id,
  nombre AS nombre_cliente,
  LOWER(TRIM(email)) AS email,
  -- LOWER: normaliza email a minusculas
  -- TRIM: elimina espacios al inicio y final
  COALESCE(ciudad, "Sin ciudad") AS ciudad,
  COALESCE(pais, "Sin pais") AS pais,
  fecha_reg::DATE AS fecha_registro,
  -- ::DATE: asegura que es tipo DATE, no TIMESTAMP
  activo
FROM {{ source('tienda', 'clientes') }}
```

-- source(): referencia a la tabla de origen declarada en sources.yml
-- No ponemos WHERE activo=TRUE aqui: staging incluye todos los datos
-- El filtro de activos va en los modelos marts donde tiene sentido

RESULTADO: dbt_dev.stg_clientes despues de dbt run

cliente_id	nombre_cliente	email	ciudad	pais	fecha_registro	act
1	Ana Torres	ana@email.com	Lima	Peru	2024-01-01	TRUE
2	Luis Garcia	luis@email.com	Bogota	Colombia	2024-01-05	TRUE
8	Juan Perez	juan@email.com	Lima	Peru	2024-02-20	TRUE

Casi identico al origen, pero email normalizado, fecha casteada y NULLs reemplazados.

```
-- Archivo: models/staging/stg_pedidos.sql

SELECT
  pedido_id,
  cliente_id,
  fecha::TIMESTAMP AS fecha_pedido,
  fecha::DATE AS fecha_pedido_date,
  LOWER(TRIM(estado)) AS estado,
  -- Normaliza el estado a minusculas y sin espacios
  total::DECIMAL(12,2) AS total,
  EXTRACT(YEAR FROM fecha)::INT AS anio_pedido,
  EXTRACT(MONTH FROM fecha)::INT AS mes_pedido
  -- Pre-calcular anio y mes aqui ahorra repetir EXTRACT en cada mart
FROM {{ source('tienda', 'pedidos') }}
```

```
-- Archivo: models/staging/stg_productos.sql

SELECT
    producto_id,
    nombre AS nombre_producto,
    LOWER(TRIM(categoria)) AS categoria,
    precio::DECIMAL(10,2) AS precio,
    stock,
    activo
FROM {{ source('tienda', 'productos') }}
```

```
-- Archivo: models/staging/stg_detalle.sql

SELECT
    detalle_id,
    pedido_id,
    producto_id,
    cantidad::INT AS cantidad,
    precio_unit::DECIMAL(10,2) AS precio_unit,
    (cantidad * precio_unit)::DECIMAL(12,2) AS subtotal
    -- Pre-calcular subtotal aqui - se usa en multiples marts
FROM {{ source('tienda', 'detalle_pedidos') }}
```

5.4 Capa Intermediate — combinar y preparar

```
-- Archivo: models/intermediate/int_pedidos_completos.sql
-- Combina pedidos con informacion del cliente y calcula el total real

SELECT
    p.pedido_id,
    p.cliente_id,
    c.nombre_cliente,
    c.ciudad,
    c.pais,
    p.fecha_pedido_date AS fecha,
    p.estado,
    p.total,
    p.anio_pedido,
    p.mes_pedido,
    COUNT(d.detalle_id) AS num_lineas_pedido,
    SUM(d.subtotal) AS total_calculado
    -- total_calculado: suma real de los detalles (debe coincidir con total)
FROM {{ ref('stg_pedidos') }} p
JOIN {{ ref('stg_clientes') }} c ON p.cliente_id = c.cliente_id
JOIN {{ ref('stg_detalle') }} d ON p.pedido_id = d.pedido_id
GROUP BY p.pedido_id, p.cliente_id, c.nombre_cliente, c.ciudad,
        c.pais, p.fecha_pedido_date, p.estado, p.total, p.anio_pedido, p.mes_pedido
```

RESULTADO: dbt_dev.int_pedidos_completos

pedido_id	nombre_cliente	ciudad	fecha	estado	total	total_ca
1	Ana Torres	Lima	2024-01-15	completado	2545.00	2545.00
2	Luis Garcia	Bogota	2024-01-20	completado	164.00	164.00
3	Ana Torres	Lima	2024-02-01	completado	650.00	650.00

6	Sofia Mendez	Buenos Aires	2024-03-01	completado	2620.00	2620.00
---	--------------	--------------	------------	------------	---------	---------

total y total_calculado deben coincidir siempre. Si difieren, hay un error en los datos. Los tests van a verificar esto automaticamente.

5.5 Capa Marts — los modelos finales que se consultan

```
-- Archivo: models/marts/ventas_por_mes.sql

{{
  config(materialized="table")
}}

SELECT
  DATE_TRUNC('month', fecha)::DATE AS mes,
  COUNT(DISTINCT pedido_id)        AS num_pedidos,
  COUNT(DISTINCT cliente_id)       AS clientes_unicos,
  SUM(total)                       AS ventas_totales,
  ROUND(AVG(total), 2)             AS ticket_promedio,
  CASE
    WHEN SUM(total) > 2000 THEN 'Excelente'
    WHEN SUM(total) > 500  THEN 'Bueno'
    ELSE                       'Normal'
  END AS clasificacion
FROM {{ ref('int_pedidos_completos') }}
WHERE estado = 'completado'
GROUP BY DATE_TRUNC('month', fecha)
ORDER BY mes;
```

RESULTADO: dbt_dev.ventas_por_mes

mes	num_pedidos	clientes_unicos	ventas_totales	ticket_promedio	clasificacion
2024-01-01	2	2	2709.00	1354.50	Excelente
2024-02-01	2	2	865.00	432.50	Bueno
2024-03-01	2	2	2695.00	1347.50	Excelente
2024-04-01	1	1	245.00	245.00	Normal

```
-- Archivo: models/marts/resumen_clientes.sql

{{
  config(materialized="table")
}}

WITH base AS (
  SELECT * FROM {{ ref('stg_clientes') }} WHERE activo = TRUE
),
pedidos AS (
  SELECT
    cliente_id,
    COUNT(pedido_id)        AS total_pedidos,
    COALESCE(SUM(total), 0) AS total_gastado,
    MAX(fecha)              AS ultimo_pedido
  FROM {{ ref('int_pedidos_completos') }}
  WHERE estado = 'completado'
  GROUP BY cliente_id
)
SELECT
  b.cliente_id,
  b.nombre_cliente,
  b.ciudad,
  b.pais,
  b.fecha_registro,
```

```

        COALESCE(p.total_pedidos, 0) AS total_pedidos,
        COALESCE(p.total_gastado, 0) AS total_gastado,
        p.ultimo_pedido,
        CASE
            WHEN COALESCE(p.total_gastado,0) >= 1000 THEN 'VIP'
            WHEN COALESCE(p.total_gastado,0) >= 200 THEN 'Regular'
            WHEN COALESCE(p.total_gastado,0) > 0 THEN 'Ocasional'
            ELSE 'Sin compras'
        END AS segmento
FROM base b
LEFT JOIN pedidos p ON b.cliente_id = p.cliente_id
ORDER BY total_gastado DESC

```

```

-- Archivo: models/marts/top_productos.sql

{{ config(materialized="table") }}

SELECT
    pr.producto_id,
    pr.nombre_producto,
    pr.categoria,
    pr.precio,
    COALESCE(SUM(d.cantidad), 0) AS unidades_vendidas,
    COALESCE(SUM(d.subtotal), 0) AS ingresos_totales,
    RANK() OVER (ORDER BY COALESCE(SUM(d.subtotal),0) DESC) AS ranking
FROM {{ ref('stg_productos') }} pr
LEFT JOIN
    (SELECT d.* FROM {{ ref('stg_detalle') }} d
     JOIN {{ ref('int_pedidos_completos') }} p
       ON d.pedido_id = p.pedido_id AND p.estado = 'completado'
    ) d ON pr.producto_id = d.producto_id
WHERE pr.activo = TRUE
GROUP BY pr.producto_id, pr.nombre_producto, pr.categoria, pr.precio
ORDER BY ingresos_totales DESC

```

```

-- Archivo: models/marts/rfm_segmentacion.sql

{{ config(materialized="table") }}

WITH rfm_base AS (
    SELECT * FROM {{ ref('resumen_clientes') }}
    WHERE total_pedidos > 0
),
rfm_scores AS (
    SELECT *,
        4 - NTILE(3) OVER (ORDER BY
            CURRENT_DATE - ultimo_pedido::DATE DESC) AS r_score,
        NTILE(3) OVER (ORDER BY total_pedidos) AS f_score,
        NTILE(3) OVER (ORDER BY total_gastado) AS m_score
    FROM rfm_base
)
SELECT
    cliente_id, nombre_cliente, ciudad,
    total_pedidos, total_gastado, ultimo_pedido,
    r_score, f_score, m_score,
    r_score + f_score + m_score AS rfm_total,
    segmento AS segmento_valor,
    CASE WHEN r_score+f_score+m_score >= 8 THEN
        'VIP-RFM'
        WHEN r_score+f_score+m_score >= 5 THEN
        'Regular-RFM'
        ELSE 'En Riesgo-RFM' END AS segmento_rfm

```

```
FROM rfm_scores ORDER BY rfm_total DESC
```

6. Tests en dbt — validar datos automaticamente

Los tests son la característica mas valiosa de dbt para producción. Ejecutas dbt test y dbt verifica automaticamente que tus datos cumplen las reglas.

6.1 Tests integrados — los 4 que siempre usas

LOS 4 TESTS NATIVOS DE DBT

Test	Que verifica	Ejemplo de uso	Cuando falla
not_null	Que la columna no tiene valores NULL	cliente_id never NULL	Si hay una fila con cliente_id = NULL
unique	Que todos los valores de la columna son distintos	email es unico en stg_clientes	Si dos filas tienen el mismo email
accepted_values	Que los valores estan dentro de una lista permitida	estado solo puede ser completado, enviado, pendiente, cancelado	Si aparece estado = 'devuelto' que no esta en la lista
relationships	Que cada valor FK existe en la tabla referenciada	cliente_id en pedidos existe en clientes.cliente_id	Si hay un pedido con cliente_id = 99 que no existe en clientes

```
# Archivo: models/staging/schema.yml
# Aqui declaras los tests para los modelos de staging

version: 2

models:
  - name: stg_clientes
    description: "Clientes normalizados desde tienda.clientes"
    columns:
      - name: cliente_id
        description: "PK del cliente"
        tests:
          - not_null
          - unique
          # cliente_id nunca puede ser NULL ni repetirse
      - name: email
        description: "Email normalizado a minusculas"
        tests:
          - not_null
          - unique
      - name: activo
        tests:
          - not_null
          - accepted_values:
              values: [true, false]
```

```

- name: stg_pedidos
  columns:
    - name: pedido_id
      tests:
        - not_null
        - unique
    - name: cliente_id
      tests:
        - not_null
        - relationships:
            to: ref("stg_clientes")
            field: cliente_id
            # Verifica que cada cliente_id en pedidos existe en stg_clientes
- name: estado
  tests:
    - accepted_values:
        values: ["completado", "enviado", "pendiente", "cancelado"]
- name: total
  tests:
    - not_null

```

```

# Ejecutar los tests:
dbt test

# Resultado esperado (todos pasan):
# 14:32:01 Running with dbt=1.7.x
# 14:32:02 Found 8 models, 12 tests
# 14:32:03
# 14:32:03 Concurrency: 4 threads (target="dev")
# 14:32:03
# 14:32:04 1 of 12 START test not_null_stg_clientes_cliente_id ...
# 14:32:04 1 of 12 PASS not_null_stg_clientes_cliente_id
# 14:32:04 2 of 12 START test unique_stg_clientes_cliente_id ...
# 14:32:04 2 of 12 PASS unique_stg_clientes_cliente_id
# ...
# 14:32:06 Finished running 12 tests
# 14:32:06
# 14:32:06 Completed successfully
# 14:32:06
# 14:32:06 Done. PASS=12 WARN=0 ERROR=0 SKIP=0 TOTAL=12

# Si un test falla:
# 14:32:04 1 of 12 FAIL 2 unique_stg_clientes_email
# (hay 2 clientes con el mismo email)

# Ver las filas que fallaron:
dbt test --select stg_clientes --store-failures
# Guarda las filas que fallan en una tabla dbt_dev.failures...

```

6.3 Ver las filas que fallaron — --store-failures en detalle

Cuando un test falla, dbt te dice cuantas filas fallaron pero no cuales. Con --store-failures, guarda exactamente las filas problematicas en una tabla para que puedas analizarlas.

```

# Ejecutar tests y guardar las filas que fallan:
dbt test --store-failures
# Crea tablas en el schema dbt_dev con prefijo: dbt_test__audit

# Para un test especifico:

```



```

dbt test --select stg_clientes --store-failures

# Ver las tablas de auditoria creadas:
-- En psql:
\dt dbt_dev.dbt_test__audit*

-- Ejemplo de tablas que crea:
-- dbt_dev.dbt_test__audit.not_null_stg_clientes_cliente_id
-- dbt_dev.dbt_test__audit.unique_stg_clientes_email
-- dbt_dev.dbt_test__audit.relationships_stg_pedidos_cliente_id

-- Ver exactamente que filas fallaron en unique(email):
SELECT * FROM "dbt_dev"."dbt_test__audit"."unique_stg_clientes_email";
-- Devuelve las filas duplicadas con su email y cuantas veces aparece

-- Ejemplo de lo que veria si hubiera 2 clientes con el mismo email:
-- email | n
-- -----|--
-- ana@email.com | 2  <- este email aparece 2 veces -> test falla

-- Ver filas con relationship roto (FK que no existe):
SELECT * FROM "dbt_dev"."dbt_test__audit"."relationships_stg_pedidos_cliente_id";
-- Devuelve los pedidos con cliente_id que no existe en stg_clientes

```

FLUJO COMPLETO CUANDO UN TEST FALLA

Paso	Comando	Que ves	Que haces
1. Ejecutar tests con store-failures	dbt test --store-failures	FAIL 2 unique_stg_clientes_email (2 filas fallaron)	Sabes que hay 2 duplicados pero no cuales
2. Ver las filas problematicas	SELECT * FROM dbt_test__audit.unique_stg_clientes_email	email=ana@email.com aparece 2 veces con cliente_id 1 y 9	Sabes exactamente que corregir en los datos de origen
3. Corregir el problema	UPDATE tienda.clientes SET email=... WHERE cliente_id=9	(corriges el dato duplicado)	El problema de datos esta resuelto
4. Volver a correr los tests	dbt run && dbt test	PASS unique_stg_clientes_email	Todo verde. El pipeline esta limpio.

Las tablas de auditoria se recrean en cada dbt test --store-failures. No son permanentes — son para depuracion.

RESULTADO de dbt test con los datos del curso

Test	Modelo	Resultado	Por que
not_null(cliente_id)	stg_clientes	PASS	Todos los 8 clientes tienen ID
unique(cliente_id)	stg_clientes	PASS	Cada ID es unico
not_null(email)	stg_clientes	PASS	Todos tienen email
unique(email)	stg_clientes	PASS	Todos los emails son distintos
not_null(pedido_id)	stg_pedidos	PASS	Los 10 pedidos tienen ID

unique(pedido_id)	stg_pedidos	PASS	Cada pedido_id es unico
relationships(cliente_id)	stg_pedidos	PASS	Todos los cliente_id existen en clientes
accepted_values(estado)	stg_pedidos	PASS	Solo hay: completado, enviado, pendiente, cancelado

6.2 Test personalizado — logica de negocio compleja

```
-- Archivo: tests/assert_total_coincide_con_detalle.sql
-- Este test verifica que el total del pedido coincide con la suma de sus detalles
-- Si devuelve ALGUNA fila, el test FALLA

SELECT
    pedido_id,
    total,
    total_calculado,
    ABS(total - total_calculado) AS diferencia
FROM {{ ref('int_pedidos_completos') }}
WHERE ABS(total - total_calculado) > 0.01
-- Si hay diferencias de mas de 1 centavo, algo esta mal
-- Un test personalizado debe devolver 0 filas para pasar
```

7. Documentacion — dbt docs generate

dbt genera automaticamente un sitio web con documentacion completa de todos tus modelos, sus columnas, sus tests y el linaje visual. Sin escribir nada extra.

7.1 Agregar descripciones en el schema.yml

```
# Archivo: models/marts/schema.yml

version: 2

models:
  - name: ventas_por_mes
    description: >
      Ventas agregadas por mes calendario. Solo incluye pedidos con estado
      completado. Usado por el dashboard ejecutivo semanal.
    columns:
      - name: mes
        description: "Primer dia del mes (DATE)"
      - name: num_pedidos
        description: "Cantidad de pedidos completados en el mes"
      - name: ventas_totales
        description: "Suma de todos los totales de pedidos completados"
      - name: clasificacion
        description: >
          Excelente si ventas > 2000, Bueno si ventas > 500, Normal en otro caso.
  - name: resumen_clientes
    description: "Un registro por cliente activo con su historial de compras y segmento"
    columns:
      - name: segmento
        description: "VIP (>= 1000), Regular (>= 200), Ocasional (> 0), Sin compras"
    tests:
      - accepted_values:
          values: ["VIP", "Regular", "Ocasional", "Sin compras"]
```

7.2 Generar y ver la documentacion

```
# Generar la documentacion:
dbt docs generate
# Lee todos los schema.yml y genera un sitio estatico en target/docs/

# Abrir la documentacion en el navegador:
dbt docs serve
# Abre: http://localhost:8080

# Lo que puedes ver en la interfaz web:
# - Lista de todos los modelos con sus descripciones
# - Para cada modelo: columnas, tipos, tests, descripcion
# - El grafo de linaje: que tablas usa cada modelo y quien lo usa a el
# - Resultados de los tests

# Para ver el linaje de un modelo especifico:
# 1. Abre dbt docs serve
# 2. Busca "ventas_por_mes" en la barra de busqueda
# 3. Click en "View Lineage Graph"
# 4. Ves el grafo: tienda.pedidos -> stg_pedidos -> int_pedidos_completos -> ventas_por_mes
```

7.3 El grafo de linaje — como leerlo

El linaje es el grafico que muestra de donde viene cada dato. En dbt docs serve puedes verlo de forma interactiva. Aqui tienes la version completa en texto para entender la estructura antes de abrir el navegador.

GRAFO DE LINAJE COMPLETO — proyecto tienda_dbt

Nivel	Nodo	Tipo	Depende de	Lo usan
0 — Origen	tienda.clientes	source (tabla real)	(ninguno)	stg_clientes
0 — Origen	tienda.pedidos	source (tabla real)	(ninguno)	stg_pedidos
0 — Origen	tienda.productos	source (tabla real)	(ninguno)	stg_productos
0 — Origen	tienda.detalle_pedidos	source (tabla real)	(ninguno)	stg_detalle
1 — Staging	stg_clientes	view	tienda.clientes	int_pedidos_completos , resumen_clientes
1 — Staging	stg_pedidos	view	tienda.pedidos	int_pedidos_completos
1 — Staging	stg_productos	view	tienda.productos	top_productos
1 — Staging	stg_detalle	view	tienda.detalle_pedidos	int_pedidos_completos , top_productos
2 — Intermediate	int_pedidos_completos	view	stg_clientes + stg_pedidos + stg_detalle	ventas_por_mes, resumen_clientes, top_productos
3 — Marts	ventas_por_mes	table	int_pedidos_completos	(consumo final — dashboards)
3 — Marts	resumen_clientes	table	stg_clientes + int_pedidos_completos	rfm_segmentacion
3 — Marts	top_productos	table	stg_productos + stg_detalle + int_pedidos_completos	(consumo final)
3 — Marts	rfm_segmentacion	table	resumen_clientes	(consumo final)

Si cambias tienda.pedidos: se impactan stg_pedidos -> int_pedidos_completos -> ventas_por_mes, resumen_clientes, top_productos, rfm_segmentacion. dbt te muestra esto visualmente con colores en dbt docs serve.

EL LINAJE VISUAL — que modelos dependen de que

Modelo final	Depende de (intermediate)	Que depende de (staging)	Que usa (source)
ventas_por_mes	int_pedidos_completos	stg_pedidos, stg_clientes, stg_detalle	tienda.pedidos, tienda.clientes, tienda.detalle_pedidos
resumen_clientes	int_pedidos_completos	stg_pedidos, stg_clientes, stg_detalle	tienda.pedidos, tienda.clientes, tienda.detalle_pedidos
top_productos	stg_productos, stg_detalle, int_pedidos_completos	stg_productos, stg_detalle	tienda.productos, tienda.detalle_pedidos
rfm_segmentacion	resumen_clientes	-	(via resumen_clientes)

El linaje te dice exactamente que pasa si cambias tienda.pedidos: se rompen stg_pedidos, int_pedidos_completos, ventas_por_mes, resumen_clientes y rfm_segmentacion.

8. Los comandos dbt que usas cada día

REFERENCIA RAPIDA DE COMANDOS

Comando	Que hace	Cuando usarlo
dbt run	Ejecuta TODOS los modelos en el orden correcto	Cuando quieres actualizar todas las tablas y vistas
dbt run --select staging	Ejecuta solo los modelos de la carpeta staging	Cuando cambiaste algo en staging y quieres probar solo eso
dbt run --select ventas_por_mes	Ejecuta solo el modelo ventas_por_mes	Cuando cambiaste un modelo específico
dbt run --select +ventas_por_mes	Ejecuta ventas_por_mes Y todos sus dependientes	Para reconstruir un modelo y todo lo que lo usa
dbt run --select ventas_por_mes+	Ejecuta ventas_por_mes Y todos los que DEPENDEN de el	Para verificar que nada se rompio aguas abajo
dbt test	Ejecuta todos los tests definidos en schema.yml y tests/	Antes de hacer commit — verifica que los datos son correctos
dbt test --select stg_clientes	Ejecuta solo los tests de stg_clientes	Para verificar un modelo específico rápidamente
dbt docs generate	Genera la documentacion estatica en target/docs/	Antes de compartir el proyecto con el equipo
dbt docs serve	Abre la documentacion en http://localhost:8080	Para explorar el linaje y la documentacion visualmente
dbt debug	Verifica la configuracion y la conexion a la BD	Cuando algo no funciona y no sabes por que
dbt compile	Genera los SQL compilados (con ref() y source() resueltos)	Para ver exactamente que SQL ejecutara dbt sin ejecutarlo
dbt seed	Carga archivos CSV de la carpeta seeds/ como tablas	Para datos de referencia estaticos (países, categorías, etc.)

PROYECTO 3 — Data Mart con dbt

PROYECTO FASE 3

Data Mart de Ventas con dbt Core + Tests + Documentacion

4-5 días | Junior-Mid

Que construyes

Un Data Mart completo sobre los datos de tienda con dbt. 5 modelos en 4 capas, 15+ tests automaticos, documentacion con linaje visual. El mismo SQL que escribiste en los modulos 1A y 1B, ahora versionado y profesional.

CHECKLIST DE ENTREGABLES — Proyecto 3

	Entregable	Criterio de aceptacion
☐	Proyecto dbt inicializado	dbt debug muestra All checks passed!
☐	sources.yml completo	Las 4 tablas de tienda declaradas con descripcion
☐	stg_clientes.sql	Email normalizado, NULLs manejados, dbt run sin errores
☐	stg_pedidos.sql	Estado normalizado, anio_pedido y mes_pedido calculados
☐	stg_productos.sql y stg_detalle.sql	subtotal pre-calculado en stg_detalle
☐	int_pedidos_completos.sql	Combina staging correctamente, total == total_calculado
☐	ventas_por_mes.sql (mart)	Resultado coincide con la tabla de referencia del modulo 1A
☐	resumen_clientes.sql (mart)	Segmento correcto para cada cliente
☐	top_productos.sql (mart)	Laptop en ranking 1 con 5000 de ingresos
☐	rfm_segmentacion.sql (mart)	Ana Torres como VIP
☐	Tests: 15+ en schema.yml	dbt test muestra PASS=15+ ERROR=0
☐	Test personalizado	assert_total_coincide_con_detalle.sql pasa
☐	Documentacion completa	dbt docs generate + dbt docs serve funciona
☐	Linaje visible	Captura del grafo de linaje en capturas/
☐	GitHub con commits	5+ commits descriptivos feat/fix/docs

Estructura final del repositorio

```
data-engineering-master/  
└─ tienda_dbt/  
   └─ dbt_project.yml  
   └─ models/  
      └─ staging/
```

```

├── sources.yml
├── schema.yml          <- tests de stg_clientes y stg_pedidos
├── stg_clientes.sql
├── stg_pedidos.sql
├── stg_productos.sql
├── stg_detalle.sql
├── intermediate/
│   ├── schema.yml
│   └── int_pedidos_completos.sql
├── marts/
│   ├── schema.yml      <- tests de modelos finales
│   ├── ventas_por_mes.sql
│   ├── resumen_clientes.sql
│   ├── top_productos.sql
│   ├── rfm_segmentacion.sql
│   └── hechos_ventas.sql <- modelo incremental
├── tests/
│   └── assert_total_coincide_con_detalle.sql
├── capturas/
│   ├── dbt_run_exitoso.png
│   ├── dbt_test_exitoso.png
│   └── lineage_graph.png

```

EJERCICIO 1 — Configurar el proyecto y correr los primeros modelos

- Sigue estos pasos en orden:
- 1. Instala dbt-core y dbt-postgres en un virtualenv.
 - 2. Ejecuta dbt init tienda_dbt.
 - 3. Crea ~/.dbt/profiles.yml con la conexion a curso_postgres.
 - 4. Ejecuta dbt debug — debe mostrar All checks passed!
 - 5. Crea sources.yml y el primer modelo stg_clientes.sql.
 - 6. Ejecuta dbt run --select stg_clientes.
 - 7. Verifica en psql: SELECT * FROM dbt_dev.stg_clientes;

Pistas:

- pip install dbt-core dbt-postgres en el virtualenv activo
- El profiles.yml usa: host=localhost, port=5432, user=admin, pass=masterpass2025, dbname=cursodb, schema=dbt_dev
- dbt debug para verificar la conexion antes de crear modelos
- SELECT * FROM dbt_dev.stg_clientes en psql para ver el resultado

RESULTADO ESPERADO — SELECT * FROM dbt_dev.stg_clientes

cliente_id	nombre_cliente	email	ciudad	pais	fecha_registro	act
1	Ana Torres	ana@email.com	Lima	Peru	2024-01-01	TRUE
2	Luis Garcia	luis@email.com	Bogota	Colombia	2024-01-05	TRUE
3	Maria Lopez	maria@email.com	Madrid	Espana	2024-01-10	TRUE
4	Carlos Ruiz	carlos@email.com	Lima	Peru	2024-01-15	TRUE
5	Sofia Mendez	sofia@email.com	Buenos Aires	Argentina	2024-01-20	TRUE
6	Diego Vargas	diego@email.com	Lima	Peru	2024-01-25	TRUE
7	Paula Castro	paula@email.com	Medellin	Colombia	2024-02-01	TRUE
8	Juan Perez	juan@email.com	Lima	Peru	2024-02-20	TRUE

EJERCICIO 2 — Construir todas las capas y ejecutar dbt run

Con todos los modelos creados:

1. Ejecuta `dbt run --` debe crear 8 modelos (4 staging, 1 intermediate, 4 marts).
2. Verifica en `psql` que existen las tablas/vistas en `dbt_dev`.
3. Ejecuta `SELECT * FROM dbt_dev.ventas_por_mes` y verifica que coincide con el resultado esperado.

Pistas:

- `dbt run` ejecuta todos los modelos en el orden correcto automaticamente
- Los staging y intermediate seran vistas, los marts seran tablas
- Verificar con `\dt dbt_dev.*` en `psql` para listar todos los objetos

RESULTADO ESPERADO de dbt run

Modelo	Tipo	Filas	Tiempo esperado
stg_clientes	view	8 clientes	-
stg_pedidos	view	10 pedidos	-
stg_productos	view	10 productos	-
stg_detalle	view	18 lineas	-
int_pedidos_completos	view	10 pedidos con cliente	-
ventas_por_mes	table	4 meses	< 1 seg
resumen_clientes	table	8 clientes	< 1 seg
top_productos	table	10 productos	< 1 seg
rfm_segmentacion	table	6-7 clientes con compras	< 1 seg

EJERCICIO 3 — Agregar tests y ejecutar dbt test

Agrega los tests del `schema.yml` de la seccion 6.1 para staging. Luego ejecuta `dbt test`. Debe mostrar PASS para los 15+ tests.

Bonus: agrega el test personalizado `assert_total_coincide_con_detalle.sql` y verifica que pasa.

Pistas:

- Los tests van en `models/staging/schema.yml` y `models/marts/schema.yml`
- `dbt test` ejecuta todos los tests
- `dbt test --select stg_clientes` para probar un modelo especifico
- `dbt test --store-failures` guarda las filas que fallan para analizar

PROYECTO 5 — El mismo Data Mart en BigQuery (Cloud)

PROYECTO FASE 5

Data Mart en BigQuery con dbt + Free Tier

2-3 días | Mid

Por que BigQuery

BigQuery es el data warehouse de Google Cloud que usan la mayoría de las empresas de datos en Latinoamerica. Con dbt, correr el mismo proyecto en BigQuery es cambiar el adaptador y el profiles.yml. El SQL y la estructura de carpetas son exactamente iguales.

DIFERENCIAS ENTRE POSTGRESQL LOCAL Y BIGQUERY

Aspecto	PostgreSQL (Modulo 1-2)	BigQuery (Proyecto 5)
Donde corre	Docker en tu maquina local	Servidores de Google Cloud — en la nube
Costo	Gratis (Docker)	1TB de consultas gratis por mes. 10GB storage gratis.
Escala	Gigabytes en tu laptop	Petabytes sin configuracion
Sintaxis SQL	PostgreSQL estandar	95% igual — algunas diferencias en funciones de fecha
Conexion dbt	dbt-postgres	dbt-bigquery (un adaptador diferente)
Schema	dbt_dev (schema PostgreSQL)	dataset de BigQuery (el equivalente del schema)

5.0 Limites del free tier de BigQuery — lee esto antes

BigQuery tiene un free tier generoso, pero tiene limites reales. Conocerlos antes de empezar te evita sorpresas en la factura.

LOS LIMITES DEL FREE TIER DE BIGQUERY (2024)

Recurso	Limite gratuito por mes	Que pasa si lo superas	Como evitarlo
Consultas (Query)	1 TB de datos procesados	Se cobra ~\$5 USD por TB adicional	Usar SELECT solo las columnas que necesitas (nunca SELECT *). BigQuery cobra por datos escaneados, no por numero de queries.
Almacenamiento activo	10 GB de datos almacenados	Se cobra ~\$0.02 USD por GB adicional	Para el curso con 4 tablas pequenas, nunca llegas ni cerca de

			10 GB.
Almacenamiento a largo plazo	10 GB (tablas sin modificar por 90 días)	Se cobra ~\$0.01 USD por GB	Tablas del curso: menos de 1 MB. Completamente gratis.
BigQuery ML	10 GB de datos procesados para ML	Se cobra por GB adicional	No aplica para este modulo (usamos dbt, no BigQuery ML)

Para el Proyecto 5 del curso, el consumo total sera de unos pocos MB. Absolutamente dentro del free tier. El 1TB/mes es para queries sobre tablas de millones de filas, no para datos de practica.

! Como verificar tu consumo en BigQuery

En Google Cloud Console -> Billing -> Cost Table puedes ver en tiempo real cuanto has gastado. Para el Proyecto 5 con los datos del curso, el consumo debe ser literalmente \$0.00. Si ves algun cargo, verifica que no hayas ejecutado queries sobre tablas publicas de BigQuery muy grandes por accidente.

5.1 Configurar BigQuery

```
# Paso 1: crear cuenta en Google Cloud (si no tienes)
# https://cloud.google.com -- click en "Start for free"
# Necesitas: tarjeta de credito (no cobra hasta que superes el free tier)

# Paso 2: crear un proyecto en Google Cloud Console
# https://console.cloud.google.com
# Click en "New Project" -> nombre: "tienda-dbt-proyecto"

# Paso 3: habilitar la API de BigQuery en el proyecto
# Buscar "BigQuery API" en la consola -> Enable

# Paso 4: crear una Service Account para dbt
# IAM & Admin -> Service Accounts -> Create Service Account
# Nombre: dbt-service-account
# Role: BigQuery Data Editor + BigQuery Job User
# Click en la cuenta creada -> Keys -> Add Key -> JSON
# Descarga el archivo JSON (es tu credencial)

# Guardar el archivo en un lugar seguro:
mkdir -p ~/.gcp
mv ~/Downloads/tienda-dbt-proyecto-xxxxx.json ~/.gcp/bq-dbt-creds.json
# NUNCA subas este archivo a Git

# Paso 5: instalar el adaptador BigQuery
pip install dbt-bigquery
```

5.2 Configurar profiles.yml para BigQuery

```
# Agregar el target de BigQuery en ~/.dbt/profiles.yml

tienda_dbt:
  target: dev          # sigue apuntando a dev (PostgreSQL) por defecto
  outputs:
    dev:               # tu configuracion de PostgreSQL local (ya existe)
```

```

    type: postgres
    ...

bigquery:          # nuevo target para BigQuery
  type: bigquery
  method: service-account
  project: tienda-dbt-proyecto
  # Reemplaza por tu Project ID de Google Cloud
  dataset: dbt_tienda
  # El dataset en BigQuery (equivalente al schema)
  # dbt lo crea automaticamente si no existe
  keyfile: /home/TU_USUARIO/.gcp/bq-dbt-creds.json
  # Ruta al archivo de credenciales que descargaste
  threads: 4
  timeout_seconds: 300
  location: US
  # La region donde se guardaran tus datos

# Verificar la conexion a BigQuery:
dbt debug --target bigquery
# Debe mostrar: All checks passed!

```

5.3 Subir datos de tienda a BigQuery

BigQuery es un data warehouse en la nube — no tiene conexion directa a tu PostgreSQL local. Necesitas subir los datos de tienda. La forma mas facil con dbt son los seeds.

```

# Los seeds son archivos CSV que dbt carga como tablas
# Exportar los datos de tienda desde PostgreSQL a CSV:

docker exec curso_postgres psql -U admin -d cursodb -c
"\COPY tienda.clientes TO STDOUT WITH CSV HEADER"
> tienda_dbt/seeds/clientes.csv

docker exec curso_postgres psql -U admin -d cursodb -c
"\COPY tienda.pedidos TO STDOUT WITH CSV HEADER"
> tienda_dbt/seeds/pedidos.csv

docker exec curso_postgres psql -U admin -d cursodb -c
"\COPY tienda.productos TO STDOUT WITH CSV HEADER"
> tienda_dbt/seeds/productos.csv

docker exec curso_postgres psql -U admin -d cursodb -c
"\COPY tienda.detalle_pedidos TO STDOUT WITH CSV HEADER"
> tienda_dbt/seeds/detalle_pedidos.csv

# Cargar los CSVs a BigQuery con dbt seed:
dbt seed --target bigquery
# Crea las tablas en BigQuery: dbt_tienda.clientes, dbt_tienda.pedidos, etc.

# Ahora actualiza sources.yml para BigQuery:
# Cambia schema: tienda por schema: dbt_tienda
# (los seeds van al mismo dataset que los modelos)

```

5.4 Ejecutar el proyecto completo en BigQuery

```

# Con los seeds cargados, ejecutar todo en BigQuery:
dbt run --target bigquery

```

```

# Si hay errores de sintaxis BigQuery vs PostgreSQL,
# los mas comunes son:

-- PostgreSQL: COALESCE(campo, "valor")
-- BigQuery: IFNULL(campo, 'valor') -- usa comillas simples

-- PostgreSQL: DATE_TRUNC('month', fecha)
-- BigQuery: DATE_TRUNC(fecha, MONTH) -- el orden de parametros es diferente

-- PostgreSQL: EXTRACT(YEAR FROM fecha)
-- BigQuery: EXTRACT(YEAR FROM fecha) -- igual!

-- PostgreSQL: fecha::DATE
-- BigQuery: CAST(fecha AS DATE) -- BigQuery no tiene ::

# Para manejar las diferencias de sintaxis, dbt tiene macros:
# {{ dbt.date_trunc("month", "fecha") }}
# Esta macro funciona en PostgreSQL Y BigQuery automaticamente

# Ejecutar tests en BigQuery:
dbt test --target bigquery

# Ver en BigQuery Console:
# https://console.cloud.google.com/bigquery
# Dataset: dbt_tienda
# Tablas: ventas_por_mes, resumen_clientes, top_productos, rfm_segmentacion

```

CHECKLIST PROYECTO 5 — BigQuery

	Entregable	Criterio de aceptacion
☐	Cuenta GCP creada	Acceso a console.cloud.google.com
☐	Service account con permisos correctos	dbt debug --target bigquery muestra All checks passed!
☐	Seeds cargados	dbt seed --target bigquery crea las 4 tablas en BigQuery
☐	dbt run en BigQuery	Los 8 modelos corren sin errores
☐	dbt test en BigQuery	Todos los tests pasan en BigQuery
☐	Capturas del BigQuery Console	Screenshot de los datasets y tablas en capturas/bigquery/
☐	README actualizado	Instrucciones para correr en BigQuery con --target bigquery

Ejercicios adicionales — Para ir mas alla

EJERCICIO 4 — Macro reutilizable

Crea una macro dbt llamada clasificar_ventas que reciba un campo de monto y devuelva la clasificacion (Excelente/Bueno/Normal). Usala en ventas_por_mes y en resumen_clientes para no repetir el CASE WHEN.

Pistas:

- Las macros van en la carpeta macros/ como archivos .sql
- Sintaxis: {% macro clasificar_ventas(campo) %} ... {% endmacro %}
- Para usar: {{ clasificar_ventas("SUM(total)") }} en el modelo
- dbt compile para ver el SQL generado antes de ejecutar

SOLUCION

```
-- Archivo: macros/clasificar_ventas.sql

{% macro clasificar_ventas(campo) %}
  CASE
    WHEN {{ campo }} > 2000 THEN 'Excelente'
    WHEN {{ campo }} > 500  THEN 'Bueno'
    ELSE                        'Normal'
  END
{% endmacro %}

-- Usar en ventas_por_mes.sql:
SELECT
  mes, ventas_totales,
  {{ clasificar_ventas("ventas_totales") }} AS clasificacion
FROM {{ ref('int_pedidos_completos') }}
...
```

Macro avanzada — pasar campos y condiciones dinamicas

La macro clasificar_ventas basica recibe un campo y genera el CASE WHEN. Una macro mas avanzada puede recibir el campo, el umbral y la condicion, haciendo la logica completamente reutilizable.

```
-- Macro basica (ya la tienes):
{% macro clasificar_ventas(campo) %}
  CASE
    WHEN {{ campo }} > 2000 THEN 'Excelente'
    WHEN {{ campo }} > 500  THEN 'Bueno'
    ELSE                        'Normal'
  END
{% endmacro %}

-- Uso basico:
{{ clasificar_ventas("SUM(total)") }} AS clasificacion
-- dbt compila esto a:
-- CASE WHEN SUM(total) > 2000 THEN Excelente ...

-- -----

-- Macro avanzada: umbrales configurables
```

```
{% macro clasificar_monto(campo, umbral_alto=2000, umbral_medio=500) %}
  -- campo: la columna o expresion a evaluar
  -- umbral_alto: por defecto 2000 (pero se puede cambiar)
  -- umbral_medio: por defecto 500
  CASE
    WHEN {{ campo }} > {{ umbral_alto }} THEN 'Excelente'
    WHEN {{ campo }} > {{ umbral_medio }} THEN 'Bueno'
    ELSE 'Normal'
  END
{% endmacro %}

-- Uso con valores por defecto (igual que la basica):
{{ clasificar_monto("SUM(total)") }} AS clasificacion

-- Uso con umbrales personalizados para otro negocio:
{{ clasificar_monto("precio", umbral_alto=1000, umbral_medio=200) }} AS rango_precio

-- Macro que genera un filtro reutilizable:
{% macro solo_completados(tabla_alias="p") %}
  {{ tabla_alias }}.estado = 'completado'
{% endmacro %}

-- Uso en cualquier modelo:
WHERE {{ solo_completados('p') }}
-- Se compila a: WHERE p.estado = completado
-- Si el día de mañana el estado cambia a "finalizado",
-- solo cambias la macro y se actualiza en todos los modelos
```

CUANDO VALE LA PENA CREAR UNA MACRO

Situacion	Sin macro	Con macro
El mismo CASE WHEN en 5 modelos distintos	Copiar y pegar el CASE WHEN en cada modelo. Si cambia un umbral, editar 5 archivos.	Una macro. Si cambia el umbral, editar un solo archivo. Los 5 modelos se actualizan.
La misma condicion WHERE en muchos modelos	WHERE p.estado = 'completado' repetido en cada modelo	WHERE {{ solo_completados('p') }} en cada modelo. Cambia en un lugar.
Calculos de fechas especificos del negocio	DATE_TRUNC('month', fecha)::DATE repetido en 8 modelos	{{ inicio_mes('fecha') }} en cada modelo. Consistente y mantenible.

Regla practica: si copias el mismo bloque SQL mas de 2 veces, considera una macro.

EJERCICIO 5 — Snapshot: historial de cambios de precios

Los snapshots de dbt guardan el historial de cambios de una tabla (SCD Type 2). Crea un snapshot de tienda.productos para guardar el historial de cambios de precios. Cada vez que cambie el precio de un producto, el snapshot lo registra.

Pistas:

- Los snapshots van en la carpeta snapshots/
- Usan la estrategia timestamp (si hay updated_at) o check (comparar columnas)
- dbt snapshot para ejecutar
- dbt snapshot genera columnas: dbt_scd_id, dbt_updated_at, dbt_valid_from, dbt_valid_to

SOLUCION

```
-- Archivo: snapshots/productos_snapshot.sql

{% snapshot productos_snapshot %}

{{
    config(
        target_schema='snapshots',
        unique_key='producto_id',
        strategy='check',
        check_cols=['precio', 'stock', 'activo']
        -- Cuando precio, stock o activo cambien, crea una nueva version
    )
}}

SELECT
    producto_id,
    nombre,
    categoria,
    precio,
    stock,
    activo
FROM {{ source('tienda', 'productos') }}

{% endsnapshot %}

-- Ejecutar:
dbt snapshot

-- Ver el historial:
SELECT * FROM dbt_dev.snapshots.productos_snapshot
ORDER BY producto_id, dbt_valid_from;
-- dbt_valid_to = NULL en la version actual
-- dbt_valid_to = fecha en versiones anteriores
```


CHECKLIST FINAL — Modulo 2 completo

Marca cada punto cuando puedas hacerlo sin consultar la solucion.

Conceptos y configuracion

	Criterio
☐	Entiendo la diferencia entre el ETL manual de 1B y dbt — 3 ventajas concretas
☐	Puedo instalar dbt-core, crear un proyecto y verificar la conexion con dbt debug
☐	Entiendo la diferencia entre ref(), source() y por que cada uno existe
☐	Puedo explicar las 4 capas (sources, staging, intermediate, marts) con ejemplos
☐	Entiendo las 4 materializaciones y cuando usar table vs view vs incremental

Modelos y SQL

	Criterio
☐	Puedo crear un modelo de staging que limpia y normaliza datos de origen
☐	Puedo crear un modelo intermediate que combina staging con ref()
☐	Puedo crear modelos marts con logica de negocio usando ref() encadenados
☐	Puedo crear un modelo incremental con is_incremental() y {{ this }}
☐	Puedo crear una macro reutilizable y usarla en varios modelos

Tests y documentacion

	Criterio
☐	Puedo agregar los 4 tests nativos (not_null, unique, accepted_values, relationships)
☐	Puedo crear un test personalizado en tests/ que verifica logica de negocio
☐	dbt test muestra PASS=15+ ERROR=0 sobre los datos de tienda
☐	Puedo generar y navegar la documentacion con dbt docs generate y dbt docs serve
☐	Tengo captura del grafo de linaje de al menos un modelo marts

Proyectos 3 y 5

	Criterio
☐	Proyecto 3: los 8 modelos corren con dbt run sin errores
☐	Proyecto 3: los resultados coinciden con los del Modulo 1A y 1B
☐	Proyecto 5: dbt debug --target bigquery muestra All checks passed!
☐	Proyecto 5: dbt run --target bigquery crea las tablas en BigQuery
☐	GitHub: commits organizados con mensajes feat/fix/docs descriptivos

☐	README con instrucciones para correr en local y en BigQuery
--------------------------	---

Que sigue — Modulo 3: FastAPI + Bases de Datos

- > Exponer los datos del Data Mart como una API REST profesional
- > FastAPI con SQLAlchemy ORM — la capa de acceso a datos bien estructurada
- > JWT + RBAC — autenticacion y roles para proteger los endpoints
- > Proyecto 4: API de RRHH con SLA — el endpoint que usa los datos que construiste
- > Tests de integracion con httpx + pytest — CI/CD que verifica la API

**"El SQL que escribiste en 1A y 1B ahora esta versionado,
testeado automaticamente y documentado con linaje visual.
Eso es lo que separa un data engineer junior de uno mid."**